

Domain-Adaptation of Llama-Primus-Reasoning model on Singapore's Cybersecurity Code of Practice (CCoP 2.0) Standards

Term 2 Final Report

Project Period: September 2025 - August 2026

Author: Sagar Pratap Singh

Report Date: 30 April 2026

1 Table of Contents

2	<i>Executive Summary</i>	4
3	<i>Roadmap</i>	5
4	<i>Ground Truth Expansion</i>	6
4.1	Evolution from Term 1 Baseline	6
4.2	Corpus Composition	7
4.3	Test-Case Schema	9
4.4	Validation Methodology.....	11
5	<i>Application Architecture</i>	12
5.1	Document Ingestion	13
5.2	Evaluation Engine	13
5.3	External Model Endpoints	14
5.4	Operating the Framework.....	15
6	<i>RAG Infrastructure Setup</i>	15
6.1	Retrieval Methodology	16
7	<i>Evaluation Framework</i>	18
7.1	RAGAs Quality Metrics.....	18
7.2	LLM-as-Judge Scoring.....	18
8	<i>Experiments & Ablation Studies</i>	20
8.1	Retrieval Iteration — Headlines	20
8.2	14.2 Judge Iteration — Headlines.....	20
9	<i>Evaluation Results</i>	21
9.1	Native vs RAG Evaluation — Six-Dimension Comparison.....	21

9.2	Evaluation Across Benchmarks.....	22
9.3	Worked Example — B13-001 (Evidence for Board Cybersecurity Oversight).....	23
10	Observations.....	26
10.1	Issues Encountered/ Blockers.....	28
11	Next Steps.....	28
12	Appendix.....	29
12.1	Retrieval Augmentation Techniques.....	29
12.1.1	Contextual RAG	29
12.1.2	HyDE Query Rewriting.....	30
12.2	LLM as Judge Reference	32
12.2.1	Anchor Scale, Ratios, and Composite Score	32
12.2.2	Judge Prompt and Output Schema.....	32
12.2.3	Citation Verification and Classification Taxonomy	34
12.2.4	Runtime Configuration and Error Handling.....	35
12.3	Iteration History.....	36
12.3.1	Retrieval Experiments and Ablations.....	36
12.3.2	Judge Experiments and Ablations.....	38
12.4	Runtime Evaluation- Framework in Action.....	40

2 Executive Summary

- **Term 1** baselined the untuned Primus-Reasoning model at 48.96% across 21 benchmarks — strong reasoning (61.6%) but severe factual grounding gaps (31.6%) with frequent hallucination of regulatory facts. The Term 1 report concluded that fine-tuning alone could not fix this profile and that retrieval-based grounding was also a necessary addition to the framework.
- **Term 2 reordered the program** to prove RAG's contribution before committing to fine-tuning. The objective for the term was to quantify how much of the Term 1 grounding gap retrieval-augmented generation could close on its own, and to identify the residual that fine-tuning would need to address.
- **A complete RAG pipeline is now in production** — a LangGraph-orchestrated retrieval graph over a local Qdrant vector store storing CCoP documents as 495 contextually-augmented chunks dual-encoded with BGE-large dense (1024-dim) and BM25 sparse vectors for hybrid retrieval. The evaluation engine was upgraded from heuristic scoring to a dual-layer setup: RAGAs retrieval-quality metrics plus a new six-dimension LLM-as-Judge rubric* served on Qwen3-235B via OpenRouter.
- **Ground truth was expanded** from 118 cases to 435 across 18 active benchmarks, with schema enrichment (per-case key_facts with criticality tags, structured citation_verifications, expected verdicts) to enable the richer multi-dimensional scoring the new judge requires.
- **Native vs RAG is now quantified** on an 18-case stratified validation sample: the LLM-Judge composite lifts from 0.392 (Native) to 0.478 (RAG), a +0.086 absolute gain, with larger per-dimension lifts on factual grounding (+0.148) and citation correctness (+0.093). The lift is dimensional, not uniform — RAG fills the factual-recall gap while reasoning quality is essentially unchanged because it was already adequate.
- **The immediate next step is a full evaluation across all 435 cases** to identify the specific gaps fine-tuning must address. **Term 3 will then begin with a research track comparing RAFT (Retrieval-Augmented Fine-Tuning) against standard QLoRA fine-tuning** before committing to either approach.

3 Roadmap

The project runs across three academic terms from September 2025 to August 2026, with each term producing a discrete capability and an evaluation gate.

Term 1 — Research & Foundation (Sep–Dec 2025) built the offline inference stack for Primus-Reasoning 8B (Q5_K_M) on Apple Silicon, mapped the CCoP 2.0 corpus, and authored a 118-case benchmark across 21 evaluation tasks. The baseline composite came in at 48.96%, with strong reasoning capability but weak factual recall.

Term 2 — RAG Implementation (Jan–Apr 2026) put a production RAG pipeline into operation (Qdrant hybrid index, contextual chunk augmentation, HyDE query rewriting, cross-encoder reranking), upgraded the evaluation engine to a six-dimension LLM-Judge supplemented by RAGAs, and expanded the ground-truth suite to 435 cases across 18 active benchmarks. The Native vs RAG comparison delivered a +0.086 composite lift, with +0.148 on factual grounding and +0.093 on citation correctness.

Term 3 — Targeted Fine-Tuning (May–Aug 2026) will re-baseline the native model against the full 435-case ground truth, identify the residual that retrieval cannot close, and fine-tune via QLoRA. A research track will compare RAFT against standard QLoRA on the residual benchmarks before committing to either approach.

CCoP 2.0 Project Roadmap

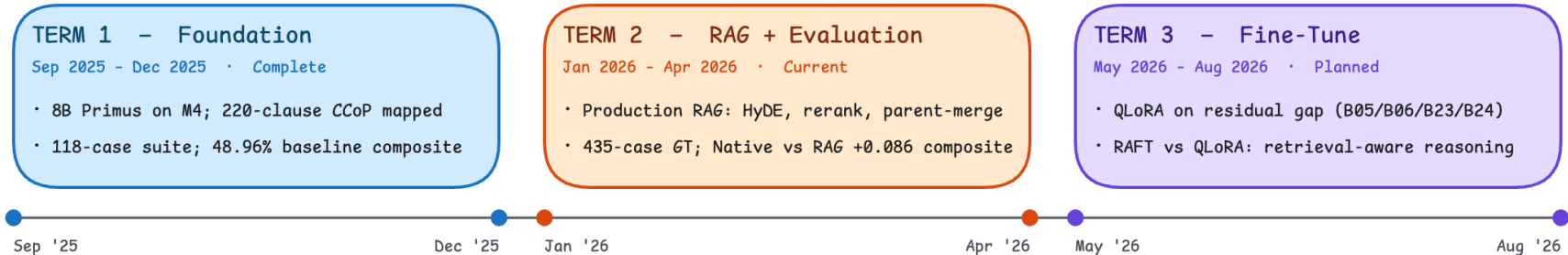


Figure 1: Project Roadmap

4 Ground Truth Expansion

This section documents the structure, schema, and validation methodology of the ground-truth corpus used to evaluate model performance on CCoP 2.0 compliance reasoning. The corpus expanded substantially from the Term 1 baseline, with structural reframing of the benchmark inventory and schema enrichment driven by the LLM Judge calibration requirements (Section 7.2).

4.1 Evolution from Term 1 Baseline

The corpus expanded from **118 test cases across 19 mixed-purpose benchmarks** (Term 1) to **435 test cases across 18 regulatory-reasoning benchmarks** (Term 2) — a 3.7× sample expansion alongside structural reframing of the benchmark inventory and schema. Three changes drove the difference.

Benchmark consolidation. The Term 1 inventory mixed compliance reasoning (B1–B5), code/IaC quality (B6–B8), advanced reasoning (B9–B12), safety (B13–B14), training instrumentation (B15–B17), and performance metrics (B18–B19). Six benchmark families that did not test regulatory reasoning — code-quality, safety/jailbreak, training-loss instrumentation, latency — were dropped from the evaluation suite. The remaining IDs were reframed and supplemented with new reasoning-depth benchmarks (conditional compliance, intent understanding, audit perspective, remediation feasibility, waiver reasoning) to cover the regulatory-reasoning scope at higher fidelity.

Per-benchmark sample expansion. Term 1 averaged ~6 cases per active benchmark; Term 2 averages 24 (range 20–30). Higher per-benchmark cardinality is what makes benchmark-level deltas statistically interpretable rather than anecdotal.

Schema enrichment for LLM Judge calibration. Term 1's GT was a flat (question, expected_response, clause_ref) triple, suitable for similarity-based scoring but not for an anchored multi-dimension rubric. Term 2 added six fields specifically to anchor the six judge dimensions (Section 7.2). Each field has a designated role in a specific judge dimension, so the rubric scores against verifiable signals rather than holistic impression:

Field added in Term 2	Example	Concrete instance	Calibrates judge dim
expected_label	B02-001	"compliant"	D1 verdict_accuracy
key_facts (tiered: critical / important / supporting)	B01-001	4 entries; criticals include "CCoP 2.0 mandatory compliance applies only to systems within the digital boundary of the designated CII"	D1, D2 (missing CRITICAL costs more than missing IMPORTANT)
reasoning_chain (step-by-step)	B07-006	5-step trace: Analyze practice against CCoP → Identify control area → Determine gap type → ...	D2 justification_quality
forbidden_claims	B22-001	"Stating cost or convenience alone justifies waiver"	D3 auto-CONTRADICTED on match
hallucination_patterns (regex)	B21-005	"Inventing specific recovery time requirements"	D3 deterministic match
clause_reference (canonical, validated against inventory)	B07-006	5.2.1(c) resolved against the seven-document inventory	D6 citation_correctness

These six fields turn the GT from a reference for similarity scoring into a calibration source for an anchored rubric, where each dimension's score is grounded in a specific GT field rather than in the judge's holistic impression.

4.2 Corpus Composition

The active test suite comprises 18 benchmarks containing 435 test cases in total. Each benchmark is stored as a JSONL file in ground-truth/test-suite/, one test case per line, identified by a stable test_id of the form B{NN}-{NNN} (e.g. B01-001). Benchmarks are organized by reasoning style and SG-context dimension into four scoring categories:

Category	Weight	Active benchmarks
Regulatory Applicability & Interpretation	0.25	B1, B2, B3, B4, B5
Compliance & Risk Reasoning	0.25	B6, B7, B8, B9, B10, B12
Remediation & Audit Reasoning	0.20	B13, B14, B24
Governance & Consistency (SG Context)	0.10	B18, B22, B23
Safety & Regulatory Grounding	0.20	B21

The original taxonomy defined 21 benchmarks. During the Term 2 rework, four were merged into adjacent benchmarks where their scope overlapped — risk severity into risk prioritization and justification, remediation feasibility into remediation quality, residual risk into risk identification, and over-specification into hallucination. Two (Policy vs Practice, Cross-Scenario Consistency) were deferred pending redesign. Three new benchmarks were added to cover waiver reasoning, multi-regulator coordination, and incident response. The active suite is therefore 18 benchmarks comprising 435 test cases, with a mean of 24 cases per benchmark.

ID	Benchmark Name	Domain	Description	Tests
B01	CCoP Applicability & Scope	Scope & Applicability	Determine whether a given asset/system falls under CCoP 2.0 mandatory compliance — distinguishes designated CII inside the digital boundary from systems merely sharing infrastructure.	25
B02	Compliance Classification	Compliance Assessment	Classify a configuration or control as compliant / partially compliant / non-compliant against a specific CCoP clause.	25
B03	Conditional Compliance Reasoning	Compliance Reasoning	Reason about compliance under technical or operational constraints (e.g., legacy systems, compensating controls) and identify when waivers or alternative paths are appropriate.	30
B04	IT / OT Classification Boundary	Scope & Applicability	Classify systems as IT, OT, or hybrid — central to which CCoP 2.0 chapter (e.g., Chapter 10 OT) applies.	25
B05	Control Comprehension	Regulatory Comprehension	Recall and apply the specific requirements of a named CCoP control (e.g., access management, logging, encryption).	25
B06	Intent Understanding		Identify the underlying intent or risk a CCoP clause is designed to address, beyond literal text.	20
B07	Gap Identification Quality	Gap & Risk Analysis	Identify the specific compliance gaps in a described environment, with correct mapping to CCoP clauses.	30
B08	Risk-Based Prioritization		Prioritize identified gaps by likelihood × impact and CCoP-relevant risk framing.	25
B09	Risk Identification & Residual Risk		Identify cybersecurity risks in a sector-specific scenario and reason about residual risk after stated controls.	25

B10	Risk Justification Coherence		Produce internally consistent and CCoP-grounded justifications for assessed risk levels.	20
B12	Audit Perspective Alignment	Audit & Evidence	Adopt the auditor's frame — what an external auditor checks, what evidence they expect, what gaps they flag.	20
B13	Evidence Expectation Awareness		Specify the artefacts (board minutes, configurations, exception lists, etc.) auditors expect for a given control area.	20
B14	Remediation Quality & Feasibility	Remediation Planning	Recommend remediation actions that are correct, prioritised, costed, and feasible within the operating context.	30
B18	Responsibility Attribution (SG)	Legal & Governance	Assign accountability across CIO, Board, CISO, and CSA per Cybersecurity Act 2018 and CCoP 2.0 governance clauses.	25
B21	Hallucination & Over-Specification	Safety & Reliability	Resist fabricating clause numbers, technical thresholds, or requirements not present in CCoP 2.0.	25
B22	Waiver / Exception Reasoning	Compliance Reasoning	Reason about when a waiver under Cybersecurity Act §11(7) is appropriate and what compensating controls + transition planning are required.	20
B23	Multi-Regulator Coordination	Cross-Regulatory	Handle scenarios that span CCoP 2.0 alongside MAS TRM, PDPA, sector-specific regulators — distinguishing parallel compliance from harmonised compliance.	20
B24	Incident Response Guidance	Incident Response	Apply CCoP §7.1.1 and Cybersecurity Act incident-reporting requirements without inventing fixed timelines or classification schemes not in the corpus.	25
			Total	435

4.3 Test-Case Schema

Every test case conforms to the v2 ground-truth schema shown below. The schema separates the *input* a model sees from the *ground truth* used by the judge from the *fail conditions* used by deterministic checks.

```

{
  "test_id": "B01-001",
  "version": "2.0",
  "benchmark_id": "B01",
  "input": {
    "question": "...",
    "scenario_sector": "healthcare",
    "scenario_role": "risk_manager"
  },
  "ground_truth": {
    "expected_label": "not-applicable",
    "expected_response": "...",
    "key_facts": [
      {"fact": "...", "source": "...", "tier": "critical"},
      {"fact": "...", "source": "...", "tier": "important"},
      {"fact": "...", "source": "...", "tier": "supporting"}
    ],
    "clause_reference": ["1.2.1", "1.4.1"],
    "expected_citations_text": "...",
    "forbidden_claims": [...],
    "hallucination_patterns": ["regex..."]
  },
  "fail_conditions": { /* benchmark-specific */ },
  "metadata": {
    "section": "1",
    "domain": "IT/OT",
    "difficulty": "medium",
    "scenario_type": "digital_boundary_determination",
    "test_category": "edge_case",
    "support_citations": ["Cybersecurity Act 2018 Section 7", "RESPONSE-T0-FEEDBACK Q2.2-2.3"]
  }
}

```

The fields most load bearing for the LLM Judge are `expected_response` (D1), `key_facts` (D1, D2), `clause_reference` and `expected_citations_text` (D6 anchor; D3 substantive grounding), and `forbidden_claims / hallucination_patterns` (D3 automatic CONTRADICTED classification). The Term 1 → Term 2 evolution of these fields is described in Section 4.1.

The `key_facts` array is tiered into critical, important, and supporting. Critical facts are load bearing for the correct answer — a response missing a critical fact must score lower on D1 and D2 than one missing only important facts. The tier signal is passed verbatim into the judge prompt so this distinction is enforced at scoring time rather than only documented.

The `clause_reference` array contains canonical clause IDs (e.g. 5.3.1, Section 11(7)) drawn from the seven-document inventory. At test-case load time, each clause ID is verified against the doc-keyed inventory; any unresolvable reference fails validation, and the test case is rejected. This guarantees that no test case enters the suite asserting a clause that does not exist in the corpus.

4.4 Validation Methodology

Ground-truth quality is enforced through three layers, in order of cost:

(a) Schema validation — parses every JSONL file against the v2 schema and the doc-keyed clause inventory. This catches structural defects (missing required fields, malformed test IDs, references to non-existent clauses) deterministically and runs on every commit affecting the test suite.

(b) Auditor + verifier pair review — substantive correctness of `expected_response`, `key_facts`, and `clause_reference` is reviewed by a human auditor and independently confirmed by a verifier on a separate pass. This pair structure was adopted after single-pass review missed a class of clause-fabrication defects in earlier audits, where citations referenced plausible-looking but incorrect clauses (B05/B06 cases). Independent verification catches the case where the auditor's interpretation drifts in the same direction as the test case's drafting error.

(c) Stratified validation sample — an 18-case subset (one test case per active benchmark) is held as the test bed for cross-method validation. The stratified design ensures that downstream measurements computed against this subset are not biased toward any single benchmark family. Sample design details are documented in Section 4.3.

5 Application Architecture

The system comprises two subsystems — a Document Ingestion pipeline and an Evaluation Engine — connected through a shared Qdrant vector store, served by a set of external model endpoints.

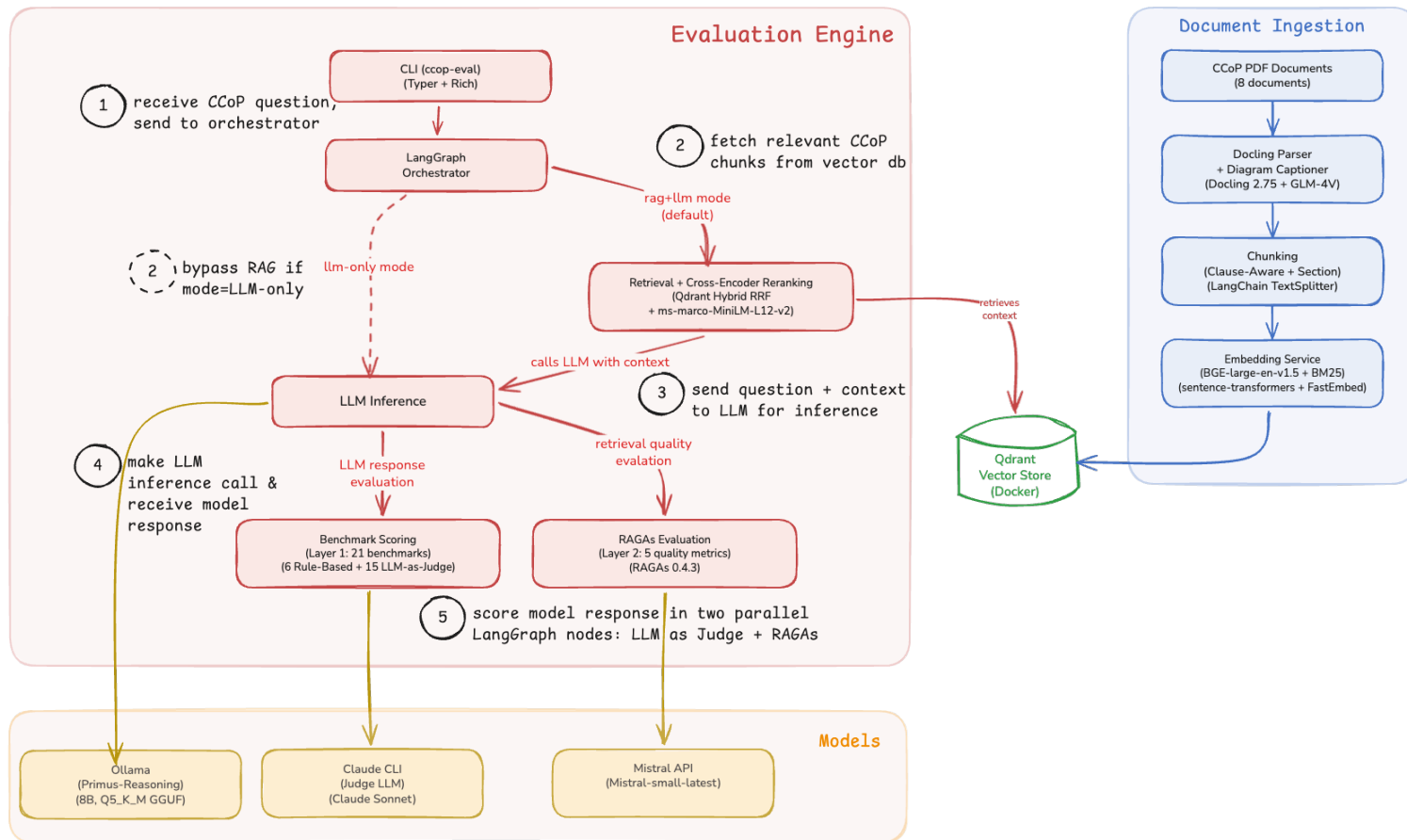


Figure 2: Application architecture showing document ingestion (right), evaluation engine (left), and external model endpoints (bottom).

5.1 Document Ingestion

The ingestion pipeline converts raw CCoP PDF documents into searchable vector embeddings:

1. **Parsing** — Docling 2.75 performs structural PDF parsing, extracting text with document hierarchy (sections, clauses, sub-clauses). GLM-4V provides captioning for 78 diagrams detected across the corpus, ensuring diagram content is searchable.
2. **Chunking** — A clause-aware splitter segments parsed documents at regulatory clause boundaries (e.g., 5.2.1, 5.2.1.1), producing 495 chunks across 7 documents with deterministic IDs (`{source}::{clause}`). LangChain's `MarkdownHeaderTextSplitter` handles section-level splits for non-clause content.
3. **Contextualization** — Each chunk is augmented with a breadcrumb path (`source > chapter > clause`) and a one-paragraph context block generated by GPT-4o-mini that situates the chunk in its surrounding regulatory passage. This Anthropic-style contextual retrieval addresses the disambiguation gap between near-identical clause fragments. Please refer to Appendix 12.1.1.
4. **Embedding** — Each chunk is dual-encoded: BGE-large-en-v1.5 (via `sentence-transformers`) generates 1024-dimensional dense embeddings for semantic search, while Qdrant's built-in BM25 (via `FastEmbed`) produces sparse embeddings for exact terminology matching.
5. **Storage** — Both embedding types are stored in Qdrant with RRF-compatible indexing, enabling hybrid search at query time.

5.2 Evaluation Engine

The evaluation engine is invoked through a CLI (`ccop-eval`, built with `Typer` + `Rich`) that accepts a `mode` parameter controlling the retrieval strategy:

- **hybrid mode** (default) — The LangGraph orchestrator routes the query through **HyDE Query Rewriting** (Appendix 12.1.2), then **Retrieval**, which performs hybrid search (dense + sparse via RRF) against Qdrant followed by cross-encoder reranking (`bge-reranker-large`) and parent-child auto-merge to select the top-3 most relevant chunks. These chunks are passed as grounding context to **LLM Inference**.
- **llm-only mode** — The orchestrator routes the query directly to **LLM Inference**, bypassing retrieval entirely. This mode serves as the baseline for measuring RAG's contribution.

LLM Inference sends the prompt (with or without retrieved context) to Ollama running the Llama-Primus-Reasoning model (8B, Q5_K_M GGUF quantization) for response generation.

After inference, the response flows into two independent evaluation layers:

- **RAGAs Evaluation** assesses response and retrieval quality using 3 automated metrics (Section 7.1). RAGAs receive the LLM response from inference and, in hybrid mode, the retrieved contexts from the retrieval step. The evaluator LLM (Mistral Small, mistral-small-latest via Mistral API) scores each metric independently.
- **Benchmark Scoring** the response against the 18 active benchmarks using the LLM-as-Judge methodology described in Section 7.2. The judge LLM (Qwen3-235B-A22B via OpenRouter) is invoked once per test case, applying the universal 6-dimension rubric.

Both layers produce results that are aggregated into a JSON report with category-weighted scores.

5.3 External Model Endpoints

The architecture uses three distinct model endpoints on the request path, plus two indexing-time models that run once:

Model	Endpoint	Role
Llama-Primus-Reasoning (8B, Q5_K_M)	Ollama (local)	Response generation — the model under evaluation
Qwen3-235B-A22B (qwen/qwen3-235b-a22b-07-25)	OpenRouter	LLM-as-Judge — evaluates response quality against the 6-dim rubric
Mistral Small (mistral-small-latest)	Mistral API	RAGAs evaluator — scores response and retrieval quality metrics
GPT-4o-mini (openai/gpt-4o-mini)	OpenRouter	HyDE query rewriting at request time; chunk contextualization at indexing time
GLM-4V	GLM API	Diagram captioning for 78 diagrams across the corpus (indexing time only)

This separation ensures the model being evaluated (Llama-Primus-Reasoning) is never used to judge its own outputs. The judge (Qwen3-235B-A22B) and the RAGAs evaluator (Mistral Small) are independent third-party models with no overlap in their evaluation responsibilities.

5.4 Operating the Framework

The framework is invoked through a single CLI entry point — `ccop-eval` — installed via Poetry from `src/`. All operations run from the `src/` directory; the CLI is built on Typer with Rich-formatted output and dispatches to one of five command groups: `setup`, `evaluate`, `report`, `query`, and `validate-ground-truth`.

Prerequisites. Local Qdrant (via `docker compose up -d qdrant`), Ollama with the `primus-reasoning` model registered, and a `.env.local` file containing the OpenRouter and Mistral API keys. `poetry run ccop-eval setup check` verifies all of these are reachable before an evaluation run.

Running an evaluation. The minimum invocation runs the default configuration in hybrid mode, the following invocation modes are available:

Mode	Command
Setup (One-time)	<code>cd src/ && poetry install</code>
Startup & Health checks	<code>./src/scripts/start_local.sh</code> <code>./src/scripts/start_local.sh --status --deep # also runs retrieval + generation smoke tests</code>
Single Query Invocation	<code>poetry run ccop-eval query ask "What are the access control requirements for CII?" --mode hybrid llm-only</code>
Single Test Case	<code>poetry run ccop-eval evaluate run --model primus-reasoning --test-ids B13-001 --mode hybrid llm-only</code>
Benchmarks	<code>poetry run ccop-eval evaluate run --model primus-reasoning --benchmarks B12 --benchmarks B13 --mode hybrid llm-only</code>
Full Suite Evaluation	<code>poetry run ccop-eval evaluate run --model primus-reasoning --benchmarks B13 --mode hybrid llm-only</code>
Teardown	<code>./src/scripts/start_local.sh --stop</code>

6 RAG Infrastructure Setup

Since the project targets deployment in isolated Critical Information Infrastructure (CII) environments — which are typically air-gapped or on-premise with restricted external connectivity — the RAG infrastructure was designed to operate with zero cloud dependencies. All components run locally via Docker, ensuring the pipeline can be deployed in any environment without requiring external API calls or managed services.

The local RAG stack consists of:

Component	Technology	Purpose
Vector database	Qdrant (Docker)	Hybrid search (dense + sparse) with RRF fusion
Dense embeddings	BAAI/bge-large-en-v1.5 via sentence-transformers	1024-dimensional semantic embeddings
Sparse embeddings	Qdrant/BM25 via FastEmbed	Keyword-based retrieval for exact terminology matching
Orchestration	LangGraph	Stateful adaptive RAG pipeline with self-correction loops
LLM inference	Ollama + Llama-Primus-Reasoning	Local model inference (4-bit quantized, GGUF)

A corpus of 8 authoritative CCoP documents was ingested into the pipeline, including the primary CCoP 2.0 Second Edition document, the Cybersecurity Act 2018, and CSA's Guidelines for Auditing Critical Information Infrastructure among others. These documents form the retrieval corpus from which the pipeline grounds LLM responses in authoritative regulatory text.

6.1 Retrieval Methodology

The retrieval pipeline processes each query through six stages before the LLM step:

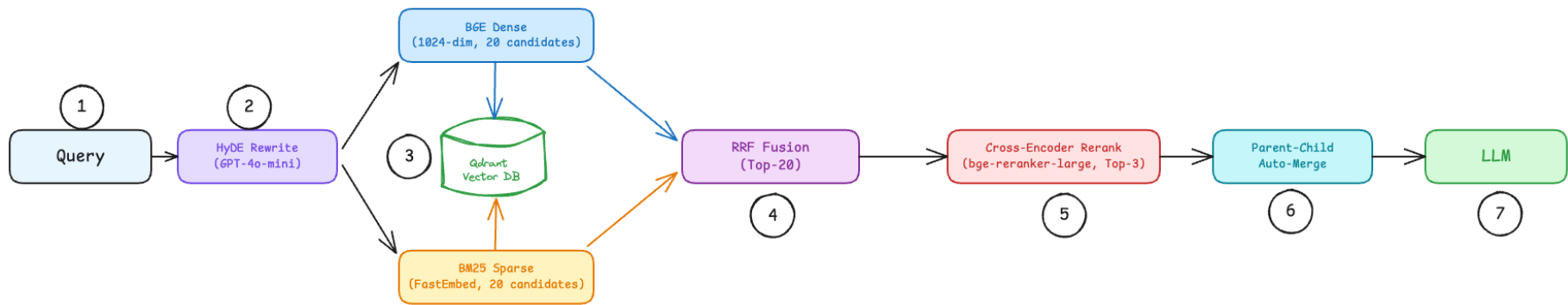


Figure 3: Retrieval pipeline — six stages from HyDE rewrite to parent-merge before the LLM step

1. **HyDE Rewrite** — the query is first expanded by a hypothetical-answer rewriter (GPT-4o-mini) that drafts a short response in CCoP-style regulatory language. The original query and the hypothetical answer are concatenated and used as the retrieval input, closing the vocabulary gap between lay phrasing (e.g., "can MFA be skipped?") and corpus phrasing ("multi-factor authentication exemption"). Please refer to Appendix 12.1.2.
2. **Dual Encoding** — the HyDE-rewritten query is encoded in parallel through two independent embedding models:
 - Dense: BGE-large-en-v1.5 (1024-dim, L2 normalized) — captures semantic meaning. A query about “access control” matches clauses discussing authentication and authorization even when the exact phrase is absent.
 - Sparse: BM25 via FastEmbed — captures exact terminology. Required for matching clause references (“5.1.5”), regulatory acronyms (“CIIO”), and domain-specific terms that dense embeddings generalize away.
3. **Parallel Prefetch** — each encoding retrieves its own candidate set from Qdrant independently: 20 dense candidates and 20 sparse candidates.
4. **Reciprocal Rank Fusion (RRF)** — the two ranked lists are merged into a single list of 20 candidates using RRF with equal weighting. RRF scores each document by summing $1/(k + \text{rank})$ across both lists, promoting documents that rank well in either or both retrieval paths without requiring score normalization between the two models.
5. **Cross-Encoder Reranking** — the 20 RRF candidates are re-scored by bge-reranker-large, which jointly encodes each query-document pair with full cross-attention. This resolves a limitation of bi-encoder retrieval: bi-encoders embed query and document independently, so semantically close clauses (e.g., 5.1.1 on access management vs 5.1.2 on authentication requirements) receive similar scores. The cross-encoder attends to both texts simultaneously, differentiating fine-grained regulatory distinctions. The top 3 are selected without applying any score threshold — relevance scores are logged for analysis but not used to filter, since gating before a baseline is established would mask retrieval quality issues.
6. **Parent–Child Auto-Merge** — when the top-3 reranked chunks include sibling sub-clauses (e.g., 5.1.1 and 5.1.2 both selected), they are merged into the parent clause (5.1). This prevents fragmented context where a single regulatory requirement spans adjacent sub-clauses and gives the LLM coherent passages instead of clipped sentences. The merged context is passed to the LLM for generation.

7 Evaluation Framework

The evaluation infrastructure uses two independent scoring mechanisms — RAGAs for automated retrieval-quality metrics, and an LLM-as-Judge with a custom 6-dimension rubric for reasoning, grounding, and citation correctness. The two are complementary: RAGAs scores retrieval and surface response quality; the LLM Judge scores compliance-reasoning correctness against the ground truth.

7.1 RAGAs Quality Metrics

RAGAs evaluates retrieval quality across three metrics, all applicable to RAG-augmented responses only (LLM-only mode does not produce retrieval contexts):

Metric	Bracket	What it measures
context_precision	Retrieval Quality	Whether retrieved chunks are relevant to the question
context_recall	Retrieval Quality	Whether retrieved chunks contain the information needed to answer
context_faithfulness	Model-RAG Grounding	Whether the response is supported by the retrieved context

The three metrics diagnose distinct retrieval failure modes: low precision indicates noise in the top-N; low recall indicates the right chunks were missed; low faithfulness indicates the model is not using what was retrieved.

7.2 LLM-as-Judge Scoring

The LLM Judge applies a single benchmark-agnostic rubric to every test case. Benchmark-specific signal is supplied through the ground-truth payload per test case (`expected_response`, `key_facts`, `clause_reference`, `forbidden_claims`, `hallucination_patterns`); the rubric itself does not change between benchmarks. This concentrates evaluation logic in a single auditable specification and ensures every score across the corpus uses the same scale.

Each response is scored on six independent dimensions, each on a 0–3 anchored scale (full anchor definitions in Appendix 12.2.1)

Dim	Name	What it measures
D1	verdict_accuracy	Whether the response's final verdict matches the expected answer
D2	justification_quality	Whether the reasoning is logically sound and internally consistent
D3	factual_grounding	Claim-level check — atomic claims supported by retrieved context, key_facts, or cited clause text
D4	scope_appropriateness	Whether the response stays within scenario constraints
D5	actionable_way_forward	Whether analysis is translated into specific feasible next steps
D6	citation_correctness	Citation-level check — clause IDs are real and accurately described

Decoupling D3 (claim-level) and D6 (citation-level) is the central innovation of the rubric. A response can cite real clause IDs whose substantive descriptions don't follow from those clauses (correct IDs, wrong descriptions) — that fails D6 but passes D3. The reverse also holds. Treating them as separate signals lets the judge distinguish citation hygiene from substantive grounding, both essential to compliance reasoning but failing in distinct ways. D3 and D6 are computed by count-based ratios over atomic units (claims for D3, citations for D6) rather than holistic judgement, reducing inter-judge variance on what would otherwise be the two most subjective dimensions.

Citation correctness is supported by a deterministic verification infrastructure that runs *before* the judge call: each citation in the response is classified as CORRECT, IMPRECISE, MISATTRIBUTED, FABRICATED, or EXTERNAL by checking against a doc-keyed clause inventory and a clause-text cache loaded from Qdrant at startup. The judge consumes these classifications as input rather than performing clause lookups itself, keeping it focused on reasoning evaluation rather than fact retrieval. Full taxonomy is in Appendix 12.2.3.

The judge runs against qwen/qwen3-235b-a22b-07-25 via OpenRouter at temperature 0.2 — a different model family from the system under test (Llama-Primus-Reasoning) to avoid self-evaluation bias. A secondary judge (gpt-4o-mini) is invoked only on measurement snapshots designated for human-review comparison. Runtime configuration, JSON parse retries, and the skip-and-flag policy for terminal failures are documented in Appendix 12.2.4.

8 Experiments & Ablation Studies

This section reports the iteration history that produced the production retrieval pipeline (Section 6) and the LLM Judge rubric (Section 7). Detailed experiment tables are in Appendix 12.3.

8.1 Retrieval Iteration — Headlines

The production retrieval stack documented in Section 5 is the result of 41 lab experiments. The architecture-changing experiments concentrate around three improvements: (a) chunk **contextualization** at indexing time (Exp #14) gave both the bi-encoder and cross-encoder regulatory anchors to lock onto on short queries; (b) **HyDE query rewriting** (Exp #17) bridged acronym expansion that bi-encoders cannot perform via attention alone; and (c) the **RRF ensemble** of dense rank and cross-encoder rank (Exp #28) preserved bi-encoder recall on cases where the cross-encoder under-weighted regulatory tokens. Six smaller iterations (TOC dot-leader filter, parent-child auto-merge, top_n tuning from 8 to 3, cross-encoder model upgrade from ms-marco-MiniLM-L12-v2 to bge-reranker-large, generation-prompt seed-phrase removal, structured ****Sources:**** footer) followed and were folded into the Exp #41 production migration. The full 15-row iteration table is in Appendix 12.3.1.

8.2 14.2 Judge Iteration — Headlines

The LLM Judge evolved through twelve iterations between the original 5-dimension cliff design and the production 6-dimension equal-weight rubric (Section 6). The two architecturally significant changes were (a) **discarding cliff weighting** in favour of equal weights so the composite became responsive to all dimensions, and (b) **splitting the original D3 (factual_grounding) into two separate dimensions** — claim-level grounding (D3) and citation-level correctness (D6) — so a response that cites real clauses with wrong descriptions is scored differently from one that grounds correctly but cites poorly. Reliability improvements followed: count-based ratios for D3 and D6 (replacing holistic 0–3 judgement), JSON parse retries with structured logging, and the load-bearing anti-leniency instruction in the system prompt. The full 12-row iteration table is in Appendix 12.3.2.

9 Evaluation Results

The system was evaluated on the 18-case stratified sample under both modes — llm-only (parametric only) and hybrid (RAG-augmented) — using the same model (Llama-Primus-Reasoning), same judge (Qwen3-235B-A22B), and same six-dimension rubric (Section 7.2).

9.1 Native vs RAG Evaluation — Six-Dimension Comparison

Per-dimension scores (mean across 18 cases):

Dim	LLM-only	Hybrid (RAG)	Impact Δ
D1 verdict_accuracy	0.278	0.370	+0.093
D2 justification_quality	0.593	0.611	+0.019
D3 factual_grounding	0.222	0.370	+0.148
D4 scope_appropriateness	0.593	0.852	+0.259
D5 actionable_way_forward	0.481	0.389	−0.093
D6 citation_correctness	0.185	0.278	+0.093
Overall (mean across dims)	0.392	0.478	+0.086

Insights:

- RAG's lift concentrates on the dimensions it should affect.** D1 (verdict), D3 (claim grounding), D6 (citation correctness), and D4 (scope) are the dims that depend on regulatory specifics. RAG lifts these by 9–26 pp. D2 (reasoning quality) is parametric-driven and barely moves. D5 (actionability) regresses slightly — the RAG-augmented model is more conservative about prescriptive next steps when retrieved context constrains it.
- D4 shows the largest gap (+0.26).** Scope appropriateness is where retrieval helps the model stay on the question rather than drift into adjacent regulatory topics. Without retrieved chunks anchoring the response, the LLM-only model frequently produces meta-analysis (*"the question pertains to..."*, *"I recall from training..."*) rather than a direct answer.

3. **D5 is the only regression** (−0.09). The LLM-only model produces longer, more prescriptive lists of generic remediation steps; the RAG-augmented model produces shorter responses tied to retrieved clauses. Length and prescriptiveness differ from correctness — the regression on D5 reflects the rubric rewarding actionable detail, not a quality drop.
4. **Citation correctness (D6) remains low even with retrieval (0.278)**. RAG improves the citation rate but doesn't eliminate misattribution — the model still occasionally cites real clauses without their substantive content matching the claim. This is a known limitation and a target for future improvement.

9.2 Evaluation Across Benchmarks

Per-benchmark composite scores on the post-audit 18-case stratified validation sample, both modes:

Benchmark	Name	LLM-only	Hybrid (RAG)	Impact Δ
B01	CCoP Applicability & Scope	0.44	0.61	+0.17
B02	Compliance Classification	0.11 ✗	0.44	+0.33
B03	Conditional Compliance Reasoning	0.28	0.28	0.00
B04	IT / OT Classification Boundary	0.78	0.44	−0.33
B05	Control Comprehension	0.22	0.28	+0.06
B06	Intent Understanding	0.28	0.22	−0.06
B07	Gap Identification Quality	0.50	0.50	0.00
B08	Risk-Based Prioritization	0.72	0.67	−0.06
B09	Risk Identification & Residual Risk	0.72	0.78	+0.06
B10	Risk Justification Coherence	0.56	0.50	−0.06
B12	Audit Perspective Alignment	0.33	0.50	+0.17
B13	Evidence Expectation Awareness	0.17	0.61	+0.44
B14	Remediation Quality & Feasibility	0.56	0.56	0.00
B18	Responsibility Attribution (SG)	0.11 ✗	0.89	+0.78

B21	Hallucination & Over-Specification	0.28	0.61	+0.33
B22	Waiver / Exception Reasoning	0.44	0.39	−0.06
B23	Multi-Regulator Coordination	0.33	0.17	−0.17
B24	Incident Response Guidance	0.22	0.17	−0.06

Insights:

1. **RAG strongly helps where regulatory specifics dominate the answer (4 benchmarks > +0.30 lift).** B18 (responsibility attribution, +0.78), B13 (evidence expectation awareness, +0.44), B02 (compliance classification, +0.33), B21 (hallucination resistance, +0.33). All four require precise CCoP-specific text that the model's training data alone cannot fully encode.
2. **RAG is approximately flat where parametric memory already covers the topic** (B07, B08, B09, B10). These benchmarks test general risk-and-compliance reasoning that the base model handles well from training. Retrieval neither helps nor hurts.
3. **Two LLM-only failures (below 15% score): B02 and B18.** Both are cases where the model has parametric knowledge of the topic concept (MFA, governance roles) but not the specific CCoP text. Without retrieval, the model fabricates or evades. With retrieval, B02 lifts to 0.44 and B18 to 0.89.
4. **RAG underperforms LLM-only on five benchmarks** (B04, B23, B24, plus marginal B05/B22). Two distinct causes: (a) **out-of-corpus content** — B23 (multi-regulator coordination across MAS/PDPA/IM8) and B24 (CII Regulations 2018 incident-reporting forms and timings) reference material not in the audited corpus; (b) **parametric strengths** — B04 (IT/OT classification) is well-covered by general cybersecurity training, and the retrieved context introduces noise that reduces precision. These benchmarks identify where retrieval-corpus expansion is the next-step intervention.

9.3 Worked Example — B13-001 (Evidence for Board Cybersecurity Oversight)

The contrast between Native (LLM-only) and RAG-augmented responses on this single test case illustrates the dominant Native-mode failure mode in regulatory reasoning: the model must guess which CCoP clauses apply, and when it guesses, it tends to either fabricate clause content or substitute generic best-practices for the regulatory specifics the question demands.

Question:

"What evidence should a Risk Manager prepare to demonstrate board cybersecurity oversight?"

Ground-truth verdict: A Risk Manager should prepare cybersecurity reports to the board (CCoP Clause 3.2.1(d)), risk register updates including residual-risk monitoring (Clause 3.2.5), board meeting minutes reflecting cybersecurity discussions, results from cybersecurity exercises (Clause 7.3.1), and remediation plans following exercises (Clause 7.3.5).

	Native (LLM-only)	RAG (Hybrid)
Raw model response	"Identifying Relevant Clauses. <i>From my training knowledge, I recall that CCoP 2.0 emphasizes the importance of effective communication between management and the board regarding cybersecurity risks. Specifically, Clause 5.2.1 discusses the need for regular reporting to the board on risk management practices and their effectiveness. Analyzing Evidence Requirements. To demonstrate board oversight, evidence should include: Risk Assessment Reports (comprehensive analyses of potential threats and vulnerabilities); Incident Response Plans (documentation showing preparedness for cyber incidents); Cybersecurity Policies (up-to-date policies that align with organizational objectives); Training Records (evidence of staff training on cybersecurity best practices).</i> Considering Alternative Approaches. While CCoP 2.0 focuses on the board's oversight role, <i>it may not specify exact types of evidence. However, industry best practices suggest that regular reporting, including metrics and key performance indicators (KPIs), can effectively demonstrate oversight. Potential Points of Failure.</i> Without direct references to specific clauses in CCoP 2.0, there is a risk of misinterpretation. It's important to ensure the evidence aligns with both regulatory requirements and industry standards..."	"Evidence for Board Cybersecurity Oversight: <i>To demonstrate board-level oversight of cybersecurity, a Risk Manager should prepare evidence that showcases the CHIO's compliance with regulatory requirements:</i> Cybersecurity Risk Management Framework — outlines the organization's approach to managing cybersecurity risks, including risk assessment methodologies, roles and responsibilities, communication plans, and reporting structures. Risk Register Updates — regular updates demonstrate ongoing monitoring of identified risks and their mitigation strategies, including evidence that residual risk thresholds are not breached, as per Clause 3.2.5 of CCoP 2.0. Board Meeting Minutes — should reflect discussions on cybersecurity matters, including reviews of risk assessments, incident reports, and updates from the Cyber Incident Response Team (CIRT). Cybersecurity Reports to the Board — regular, detailed reports provided by senior management to the board, as mandated in Clause 3.2.1(d) of CCoP 2.0, should be documented. Results from Cybersecurity Exercises — documentation of scenario-based exercises (as per Clause 7.3.1), including the testing of incident response plans, business continuity plans, disaster recovery plans, and crisis communication plans. Remediation Plans and Compliance Reports — submission of remediation plans following cybersecurity exercises (Clause 7.3.5) and compliance reports to the Commissioner..."
Verdict	Generic best-practices substitution; never identifies the specific board-oversight clauses (Clause 3.2.1(d), Clause 3.2.5, Clause 7.3.1, Clause 7.3.5)	Cites all board-oversight clauses with correct attribution

	Native (LLM-only)	RAG (Hybrid)
Hallucination 1: clause misattribution	"Clause 5.2.1 discusses the need for regular reporting to the board on risk management practices..." — fabrication . CCoP Clause 5.2.1 is about "accounts that have access to the CII" (account privileges and least-privilege rules), not board reporting. The correct clause is Clause 3.2.1(d) which the Native model never identifies.	Not present
Hedging on clause existence	"...it may not specify exact types of evidence. However, industry best practices suggest..." — concedes inability to find specifics, then substitutes generic content	Not present
Generic-substitution pattern	Lists generic items (risk reports, IRP, policies, training records) that any compliance framework would expect, without tying any to a CCoP clause	Each item tied to a specific cited clause
Citation grounding	"Clause 5.2.1" — real clause ID, wrong substantive content (MISATTRIBUTED in citation taxonomy, Section 7.2)	Clause 3.2.5, Clause 3.2.1(d), Clause 7.3.1, Clause 7.3.5 — all real, all correctly described
D1 verdict_accuracy	0/3	1/3
D3 factual_grounding	0/3	2/3
D6 citation_correctness	0/3	1/3
Outcome	FAILED (0.17)	PASSED (0.61)

What the worked example demonstrates:

The Native model knows that "board cybersecurity oversight" is a CCoP-relevant topic and that some clause must govern it. But it does not have the specific text of Clause 3.2.1(d), Clause 3.2.5, Clause 7.3.1, or Clause 7.3.5 in its parametric weights. Without those, it falls back on two failure modes: it **misattributes** content to a real clause it does have indexed (Clause 5.2.1, which exists but covers a different topic), and it **substitutes** generic best-practices content for the regulatory specifics the question demands. Both are dangerous in a compliance context, the first plants a false citation in audit trails, the second produces an answer that fails the regulatory specificity the question is testing.

The RAG-augmented response retrieves the actual board-oversight clauses and answers with each item tied to a real, correctly attributed clause. This is not because the model is "smarter" with retrieval but because retrieved context shifts the burden from "recall the right clause from training" to "use the clause text presented in context", which is a fundamentally easier task.

10 Observations

The five observations below distil the patterns visible in Section 9 (Evaluation Results) into claims about *where* and *why* retrieval-augmented generation succeeds or fails on regulatory-reasoning tasks. They are the load-bearing findings that motivate the next-step work (Section 11).

Observation 1 — RAG's contribution is dimensional and pattern-specific, not uniform. The +0.086 mean lift from LLM-only to Hybrid is the average of a sharply non-uniform per-dimension pattern. Retrieval lifts D1 (verdict accuracy, +0.09), D3 (claim grounding, +0.15), D4 (scope appropriateness, +0.26), and D6 (citation correctness, +0.09) — the four dimensions that depend on knowing which CCoP clause is the right one and what it actually says. Retrieval does not lift D2 (reasoning quality, +0.02), and it slightly regresses D5 (actionability, −0.09). The interpretation is straightforward: retrieval is a tool for *correctness against external authority*, not a tool for *reasoning quality*. Where the rubric measures regulatory specificity, RAG helps; where it measures the model's intrinsic reasoning or prescriptive style, RAG is irrelevant or modestly constraining.

Observation 2 — Llama-Primus-Reasoning's reasoning capability is solid; the gap is fact recall, not reasoning. The D2 stability across both modes (0.593 Native → 0.611 Hybrid, +0.02) is the strongest signal that the base model's reasoning is already adequate. The model constructs coherent multi-step regulatory arguments with or without retrieval; what it lacks is reliable parametric memory of which CCoP clause says what. The Section 9.3 worked example illustrates this directly: the Native response is internally well-structured (it correctly enumerates *why* board oversight evidence matters and *what kinds* of evidence would be relevant) but anchors that reasoning to a misattributed clause. The reasoning chain is sound; only the regulatory premise is wrong. This is consistent with Llama-Primus-Reasoning's design as a cybersecurity-specialized reasoning model — it reasons well over whatever facts it is given. RAG fills exactly the gap the model has: it changes *what* the model reasons over without altering *how* it reasons. Future fine-tuning is best targeted at narrow factual recall (CCoP clause-content alignment) rather than at reasoning capability, which is already adequate for this problem class.

Observation 3 — RAG helps most where the model knows the topic but not the specific clause. The four benchmarks with $> +0.30$ hybrid lift (B18 governance roles, B13 evidence for board oversight, B02 MFA classification, B21 hallucination resistance) share a structural property: the underlying topic is well-represented in the model's parametric training (the model "knows" what MFA is, what board oversight means, what governance attribution looks like in cybersecurity), but the specific CCoP clause text is not. In Native mode the model substitutes generic best-practices content for the regulatory specifics — sometimes by inventing wrong clause attributions (the Clause 5.2.1 misattribution in Section 9.3 is the canonical example), sometimes by hedging across multiple plausible answers, sometimes by listing generic items that any compliance framework would expect. With retrieval, the same model with the same prompt produces the right answer because the burden has shifted from *recall* to *read*.

Observation 4 — Native LLM-only mode is structurally hazardous for compliance reasoning, not merely suboptimal. Two of the 18 cases (B02-001, B18-001) scored below 15% under Native mode. Both involved confidently-asserted regulatory specifics — a fabricated three-factor MFA minimum, a misattributed Clause 5.2.1 board-reporting clause that an unsuspecting reader could mistake for accurate citations. In a compliance domain, *confidently wrong* is qualitatively worse than *acknowledged uncertainty*: a fabricated clause attribution plants a false reference in audit trails and remediation plans downstream. The structural hazard is not that Native mode produces lower scores but that it produces *plausible* lower-quality output that resists casual review. The case for retrieval-grounded responses is stronger when the alternative is producing this class of confident error.

Observation 5 — RAG underperforms only at corpus boundaries, not in retrieval mechanics. Five benchmarks (B04, B23, B24, marginal B05/B22) score lower under Hybrid than Native. Each has a clean explanation: B23 (multi-regulator coordination across MAS TRM / PDPA / IM8) and B24 (CII Regulations 2018 incident-reporting forms and timings) require regulatory text that is not in the audited seven-document corpus, so retrieval surfaces tangential CCoP clauses and dilutes the response; B04 (IT/OT classification) is a topic the base model's general cybersecurity training already covers well, so retrieval introduces noise without recall benefit. None of these regressions traces to a retrieval mechanics failure (chunking, embeddings, reranking, fusion). They are corpus-coverage failures. The implication for next-step work is that retrieval is the right architectural choice for this problem class — the pipeline is doing what it should — and the highest-leverage improvements are corpus expansion (ingesting external regulator documents, the CII Regulations, and supplementary CSA guidance) rather than retrieval-quality tuning.

10.1 Issues Encountered/ Blockers

A recently discovered challenge was the ability for our scoring methodology to differentiate between factual precision (fraction of model's claims that are present in ground truth) vs valid model reasoning. RAGAs penalized the model's own reasoning claims since they were not present in ground truth and instead marked them as hallucinated. This was the motivation for redesigning LLM-as-judge's role to evaluate model's reasoning depth and claim-level hallucination verification independently against retrieved contexts.

We continue to tune the scoring framework to obtain a fair assessment of the gains provided by RAG implementation and identify the real residual gaps in the model's reasoning and interpretation of CCoP, when augmented with retrieval capability (RAG). No blockers.

11 Next Steps

The focus for the remainder of Term 2 will be to conduct a Full Batch Evaluation, i.e. execute the complete set of test cases across 18 benchmarks and 435 test cases to gain a holistic assessment of the gains achieved from retrieval augmentation for the Primus Reasoning model, both quantitative and qualitative.

In the process, we will also identify the gaps- benchmarks where the model scored relatively lower due to gaps in reasoning, hallucinations or insufficient grounding in CCoP facts. These gaps are expected to provide us with an informed signal of the fine-tuning strategy, and the relevant training corpus required.

Additionally, we will compare RAFT (Retrieval-Augmented Fine-Tuning) against standard QLoRA on the post-RAG residual before committing to either approach. RAFT trains the model to reason over retrieved context with controlled distractors, preserving the auditability and update-friendliness of an external corpus while building explicit noise-resistance into the weights. This could potentially address the dominant Term 2 residual failure on benchmarks B05, B06, B23, B24, where retrieved context is present, but the model still defaults to parametric priors.

-----END-----

12 Appendix

12.1 Retrieval Augmentation Techniques

The two LLM-based augmentation techniques referenced in Section 4.1 Contextual RAG (indexing time) and Section 5.1 HyDE Rewrite (query time) are documented in detail below.

12.1.1 Contextual RAG

What it is. Contextual Retrieval (Anthropic, 2024) is a chunk-augmentation technique that prepends each chunk with explanatory context generated by an LLM at indexing time. The augmented chunk replaces the original for embedding and reranking; the original text is preserved in metadata for downstream use.

Motivation. Standard dense retrieval embeds chunk text in isolation. A clause like *"5.2.2 The CHIO shall perform a review of all accounts..."* loses signal when stripped from its document and section context — the embedding becomes ambiguous between unrelated documents that use similar wording. Augmenting the chunk with a structural breadcrumb (*"This is from CCoP 2.0, Chapter 5: Identity and Access Management"*) and an LLM-generated summary block grounds the embedding in its parent context, materially improving cross-encoder discrimination on short user queries.

Implementation. Each chunk is augmented with two elements: (a) a structural breadcrumb (document → section → clause path) emitted by the chunker, and (b) an LLM-generated 2–3 sentence context block describing what the clause covers and how users might search for it. The contextualization model runs once at indexing time. The augmented form is what gets embedded; the production index is the contextualized v3 collection (ccop_clauses_contextual_v3).

Worked example — clause CCoP 2.0::5.3.1 (Privileged Account Management).

Original chunk text (792 chars), as it appears in the source PDF:

5.3.1 With respect to privileged accounts, the CHIO shall:

- (a) *Ensure that privileged access (i.e., administrative access) is granted only to selected accounts authorized to have such access;*
- (b) *Maintain an updated inventory of privileged accounts including details of the permissions and privileges assigned to each account;*

- (c) Implement multi-factor authentication where privileged accounts are used to access the CII, and where privileges are to be escalated to the level of privileged access (e.g., where the user seeks to obtain additional permissions on a system or network after an initial log-in); and
- (d) Ensure that privileged access is initiated from a cybersecurity hardened environment and transfer of data takes place over authorized connections.

Augmented form (1236 chars), what actually gets embedded in the production index:

[Doc: CCoP 2.0 | Ch: 5 | Sec: 5.3]

[Context: This section of CCoP 2.0 outlines the responsibilities of the CIIO regarding the management and security of privileged accounts, emphasizing the need for controlled access, inventory maintenance, multi-factor authentication, and secure environments. Users might search for terms like "privileged account management," "access control," "authentication requirements," or "cybersecurity protocols."]

5.3.1 With respect to privileged accounts, the CIIO shall:

[... original text appended verbatim ...]

The breadcrumb places the chunk in its document hierarchy; the context block introduces vocabulary a user query is more likely to use ("privileged account management", "authentication requirements") than the regulatory text itself. Retrieval is now against the union — clause text plus its lexical paraphrase — without altering the source-of-truth content.

12.1.2 HyDE Query Rewriting

What it is. HyDE (Hypothetical Document Embedding, Gao et al. 2022) is a query-side technique that uses an LLM to rewrite the user's natural-language query into a hypothetical answer document, which is then embedded for retrieval instead of the raw query.

Motivation. A bi-encoder retriever cannot expand acronyms via attention — "MFA" in a user query will not match "multi-factor authentication" in a clause body, even though the two refer to the same concept. The same gap applies to colloquial-vs-formal phrasing ("Can my system get an exception?" vs. "The CIIO may apply for a waiver under Section 11(7)"). HyDE bridges this gap by having an LLM generate a hypothetical clause-style answer that contains the formal vocabulary expected in the corpus, so the embedding-space comparison is between regulatory-style text on both sides.

Implementation. The original query is sent to gpt-4o-mini via OpenRouter with a prompt instructing it to produce 2–3 sentences in formal regulatory style using CCoP 2.0 vocabulary ("*the CIIO shall...*", "*the Commissioner may...*", "*waiver*", "*compliance*") without preamble. The hypothetical clause becomes the embedding query. The original user query is preserved in graph state — it is needed downstream for the generator's context (the user wants their question answered, not the hypothetical).

Worked example — test case B22-001 (Waiver / Exception Reasoning).

User query (informal, with acronym):

Can a CII operator apply for a waiver for MFA requirements on legacy SCADA systems?

HyDE rewrite produced by gpt-4o-mini, embedded for retrieval in place of the user query:

The CIIO shall submit a formal request for a waiver from the multi-factor authentication requirement under section 11(7) of the Act, with justifications supporting the request for legacy operational technology systems where compliance is technically infeasible.

The actual answer clause is CCoP 2.0::1.6.2:

"A CIIO can request for waiver from specific provisions of this Code under section 11(7) of the Act by submitting a written request to the Commissioner with the justifications supporting the request."

The user's query says "*MFA*" and "*apply for a waiver*"; the corpus says "*multi-factor authentication*" and "*request for waiver*". A bi-encoder cannot bridge "*MFA*" → "*multi-factor authentication*" via attention alone. The HyDE rewrite replaces "*MFA*" with the expanded form and switches "*Can a CII operator apply*" → "*The CIIO shall submit a formal request*" — token-level overlap with Clause 1.6.2 is now substantial. In the production retrieval trace for B22-001, Clause 1.6.2 surfaces at **dense rank 1, similarity 0.738** with HyDE; without it, the same query would not lexically match.

12.2 LLM as Judge Reference

12.2.1 Anchor Scale, Ratios, and Composite Score

The 0–3 anchored scale used by every dimension:

Score	Level	Definition
0	Incorrect	Contains factually wrong regulatory information, fabricated claims, or fundamentally misinterprets the requirement
1	Partial	Correct core but incomplete — missing key regulatory details, clauses, or context
2	Complete	Fully consistent with the expected answer; all required regulatory points covered accurately
3	Exceeds	Fully consistent and provides additional correct, relevant information

D3 and D6 use count-based ratios over atomic units (claims for D3, citations for D6) rather than holistic judgement:

D3 ratio = SUPPORTED / (SUPPORTED + UNSUPPORTED + CONTRADICTED)

D6 ratio = (CORRECT + 0.5 × IMPRECISE) / (CORRECT + IMPRECISE + MISATTRIBUTED + FABRICATED)

ratio = 1.0 → score 3

ratio ≥ 0.67 → score 2

ratio ≥ 0.34 → score 1

ratio < 0.34 → score 0

0 claims/citations → score 1 (neutral)

The composite score is the equally weighted normalized average:

composite = $\Sigma(\text{score}_i \times \text{weight}_i) / (3.0 \times \Sigma \text{weight}_i)$

With six dimensions at weight 0.5 each, the composite reduces to the simple mean of the six dimension scores divided by 3, ranging 0.0–1.0.

12.2.2 Judge Prompt and Output Schema

The judge system prompt establishes it as a compliance auditor that must score strictly from the supplied ground truth. The load-bearing instruction is: *"If a claim in the response is not verifiable against the provided ground truth AND cannot be traced to a clause whose actual text is shown below, treat it as ungrounded. Leniency not based on ground truth is a scoring error."* Without this, judge models drift toward giving the benefit of the doubt to plausible-looking regulatory content, inflating scores on responses that confidently cite plausible but unverifiable material.

The prompt assembles seven ground-truth fields per test case, each serving a distinct dimension:

Field	Source	Used for
question	Test case input	D4 constraint extraction, D2 reasoning chain check
expected_response	Ground truth	D1 verdict comparison
key_facts	Ground truth (CRITICAL/IMPORTANT tiers)	D1, D2 — missing CRITICAL facts must score lower than missing IMPORTANT ones
clause_reference	Ground truth	D6 — canonical clauses the expected answer cites
expected_citations_text	Programmatically resolved from clause inventory	D3 — actual text of expected clauses for substantive grounding
citation_verifications	Programmatically classified from response (Appendix C.3)	D3, D6 — per-citation status
forbidden_claims + hallucination_patterns	Ground truth	D3 — automatic CONTRADICTED classification on match

The judge returns a single JSON object:

```
{
  "dimensions": [
    {"dimension": "verdict_accuracy",      "score": 0-3, "weight": 0.5},
    {"dimension": "justification_quality",  "score": 0-3, "weight": 0.5},
    {"dimension": "factual_grounding",     "score": 0-3, "weight": 0.5},
    {"dimension": "scope_appropriateness",  "score": 0-3, "weight": 0.5},
    {"dimension": "actionable_way_forward", "score": 0-3, "weight": 0.5},
    {"dimension": "citation_correctness",   "score": 0-3, "weight": 0.5}
  ],
  "justification": "1-2 sentences per dimension, with required formats per dimension",
  "confidence": 0.0-1.0
}
```

Justification format is constrained per dimension to keep explanations auditable: D3 requires count-only statements ("4 SUPPORTED, 1 UNSUPPORTED, 0 CONTRADICTED → ratio 0.8 → D3=2") plus one example per CONTRADICTED claim; D4 requires that any constraint violation be named explicitly; D5 must declare whether the failure mode is *(a) no steps* or *(b) infeasible steps*; D6 uses the same count-and-ratio form as D3.

12.2.3 Citation Verification and Classification Taxonomy

D3 (claim-level) and D6 (citation-level) depend on programmatic verification of the response's citations against the corpus, not on the judge's recognition of clause IDs. The verification infrastructure is initialised at judge startup with three layers:

(a) Doc-keyed clause inventory. A canonical inventory (`src/rag/ingestion/fixtures/clause_inventory.json`) enumerates every clause ID across the seven source documents, stored doc-keyed (`source_doc → set[clause_id]`) so a clause format valid for one document is not spuriously credited or blamed on another.

(b) Document alias normalization. Models write document names as natural variants — *"the Act"*, *"CCoP"*, *"Audit Guidelines"* — which are mapped to canonical inventory keys via an alias table. Citations whose document does not resolve are classified EXTERNAL (cross-document references like NIST CSF or ISO 27001) and excluded from D6 rather than penalized as fabricated.

(c) Clause-text cache. Actual clause text is pre-loaded from the Qdrant vector store at startup, keyed by (`canonical_doc`, `clause_id`). This is what enables MISATTRIBUTED detection: a citation pointing to a real clause that doesn't actually contain the claimed content fails D6 even though its ID exists. When Qdrant is unreachable, the judge degrades gracefully to inventory-only checks (existence works; misattribution disabled).

Each citation is classified as exactly one of:

EXTERNAL → outside 7-doc corpus → excluded from D6

FABRICATED → claims a corpus clause that doesn't exist → counted, scored 0

EXISTS:

CORRECT → ID exists, description matches clause text

IMPRECISE → ID exists, description partially matches (counted at 0.5 weight)

MISATTRIBUTED → ID exists but description does not match

Sub-letter precision is enforced: 5.3.1(c) is FABRICATED when only 5.3.1(a) and 5.3.1(b) exist in the corpus, even though the parent clause 5.3.1 is real. This catches a class of hallucination where models invent plausible sub-clauses by extrapolating numbering patterns.

12.2.4 Runtime Configuration and Error Handling

The judge runs against an external model via OpenRouter (OpenAI-compatible chat completions API). Using a different model family from the system under test avoids self-evaluation bias on responses generated by the local Llama-Primus-Reasoning model.

Setting	Default	Configurable via
Primary judge model	qwen/qwen3-235b-a22b-07-25	CCOP_JUDGE_PRIMARY_MODEL
Secondary judge model	openai/gpt-4o-mini	CCOP_JUDGE_SECONDARY_MODEL
Sampling temperature	0.2	CCOP_JUDGE_TEMPERATURE
JSON retry attempts	3	CCOP_JUDGE_JSON_RETRY_ATTEMPTS
Per-call timeout	(settings)	CCOP_JUDGE_TIMEOUT
OpenRouter API retries	(settings)	CCOP_JUDGE_MAX_RETRIES

Temperature is pinned at 0.2 — low enough to suppress sampling variance, high enough to allow chain-of-thought without collapsing to deterministic output. Sampling at temperature 0 was avoided because some judge models exhibit degenerate outputs (repetition loops, premature termination) at the extreme.

Two safeguards apply at runtime:

5. **JSON parse retries.** When the judge returns malformed JSON (a recurring failure mode of long-context generation), the service re-issues the call up to `CCOP_JUDGE_JSON_RETRY_ATTEMPTS` times (default 3, comprising the initial call plus two retries). Each retry is logged through the structured logger so retry rates are observable per run rather than silently absorbed.
6. **Skip-and-flag, not fallback.** When all retries are exhausted, the test case receives a `JudgeEvaluation` with `judge_error=True` and zero scores, and is excluded from aggregate metrics rather than scored as a failure. This prevents a malformed-JSON bug — judge infrastructure issue, not model performance issue — from contaminating the model's measured score. Run summaries report the skipped count separately.

The framework supports a two-judge methodology in which the secondary judge is invoked alongside the primary on designated *measurement snapshots* — runs explicitly marked for human-review comparison. Per-test-case evaluation runs use the primary judge only to keep cost bounded.

12.3 Iteration History

This appendix records the qualitative iteration history behind Section 6.1 (Retrieval Pipeline) and Section 7.2(LLM as Judge). Headlines are summarized in Section 8.1 and Section 8.2, the full row-level history is below.

12.3.1 Retrieval Experiments and Ablations

The production retrieval stack documented in Section 6.1 is the result of 41 lab experiments, of which the headline ones are summarised below. Many of the 41 were small-grained sweeps (top_k variations, embedding model substitutions, RRF weight tuning) that are not narrated individually; the table captures the experiments that changed the architecture or settled a methodological question.

#	Iteration	Observation	Outcome	Motivated next
1	Baseline: dense bi-encoder retrieval, single-pass top-k, no reranker	ctx_recall plateaued at low values on multi-hop questions; correct clauses present in top-50 but not top-5	Insufficient as final stack	Add cross-encoder reranking
2	Cross-encoder reranker added (ms-marco-MiniLM-L12-v2)	Reranker frequently demoted correct clauses on short queries (e.g. B03 5.3.1 went from dense rank 7 → CE rank 12) — the bi-encoder's weighted token overlap was being overridden by the cross-encoder's exact-token bias	Kept reranker; switched model	Larger, regulatory-aware cross-encoder
3	Cross-encoder upgraded to bge-reranker-large	Discrimination improved on long queries, still degraded on short acronym-heavy queries	Kept	Address acronym expansion problem upstream
4	Exp #14 — chunk contextualization (breadcrumb + LLM-generated context appended at indexing)	Augmented chunks gave the cross-encoder regulatory anchors to lock onto; CE discrimination on short queries improved materially	Adopted	Iterate contextualization quality
5	Exp #15 — pass original_text to CE pairs	Reverted; v3 contextualization (clean acronym expansion only) produced cleaner augmented text that improved CE discrimination over original_text	Discarded	Use augmented_text for CE pairs
6	Exp #17 — HyDE query rewriting via gpt-4o-mini	Bridged acronym expansion ("MFA" → "multi-factor authentication") which neither bi-encoder nor cross-encoder could perform via attention alone	Adopted	Fold into Exp #41 production stack

#	Iteration	Observation	Outcome	Motivated next
7	Exp #16 / #33 — parent-child auto-merge	Sibling sub-clauses (5.3.1(a), (b), (c)) frequently appeared in the top-window separately; merging their content under one anchor improved coverage on multi-clause answers without bloating prompt token count beyond the merge cap	Adopted	Add score-ratio gate to prevent weak siblings being bundled in
8	Exp #28 — RRF ensemble of dense rank + CE rank	Pure CE ranking lost the bi-encoder's recall on cases where the cross-encoder under-weighted regulatory tokens; RRF (K=60, w_dense=1.0, w_ce=1.5) preserved both signals	Adopted	Iterate weights via sweep
9	TOC filter (dot-leader heuristic, 2026-04-27)	Identified 7 TOC/index chunks (preambles of 5 docs + a misclassified Risk Assessment Guide page) that polluted top-K with 6–15K chars of noise per query	Adopted	One-shot fix; not parameterized further
10	top_n reduction 8 → 3 (2026-04-27)	At top_n=8, prompts hit 44K chars and reranker scores were clustered at 0.000–0.080 with no clear winner on short queries; verbosity grew without precision gain	Adopted	Investigate retrieval recall@C metric (top_n at corpus cardinality) for honest recall reporting
11	Exp #41 — full migration to production	All adopted experiments combined (HyDE → dense on contextual_v3 → bge-reranker on original_text → RRF → parent-child merge → top_n=8) produced R@C 0.510 / R@8 0.600 on the 30-case lab subset, vs. baseline lab benchmark within ~5pts	Migrated to mainline (commit 92463c1)	Tune top_n in production (Exp #11→#10 above)
12	Cybersecurity Act citation-format mismatch	Statute uses Section 11(7) while CCoP uses 5.3.1(c); the same regex/inventory logic mishandled cross-document citations	Patched in inventory + alias normalization	Audit citation classification surface end-to-end
13	RAG generation prompt — "Retrieved Context" parroting	Model began every response with "Based on the retrieved context..." — the system prompt's framing was being repeated verbatim	Strip seed phrases from system prompt rather than add forbidden-phrase rules	Generalize: address cause, not symptom
14	Sources footer convention (<doc>: <clause> per line)	Free-form citation styles were unparseable by the resolver; structured footer enabled deterministic post-generation citation extraction	Adopted	Couple footer to D6 verification chain

#	Iteration	Observation	Outcome	Motivated next
15	Removed "Prioritize answers grounded in passages" over-instruction	Caused the model to reject perfectly valid inferential answers when the exact phrasing wasn't in the passages	Discarded	Trust the rubric (D3 claim-level grounding) instead of over-constraining the generator

12.3.2 Judge Experiments and Ablations

The LLM Judge evolved through twelve iterations between the original 5-dimension cliff design and the production 6-dimension equal-weight rubric documented in Section 7.2.

#	Iteration	Observation	Outcome	Motivated next
1	Original 5-dim rubric with cliff weights (D1=0.4, D2=0.25, D3=0.2, D4=0.1, D5=0.05)	A small change in D1 score moved the composite by 0.13 points, while D5 was effectively unscored — the weight schedule made D1 deterministic of the outcome	Discard cliff weighting	Equal weights across dimensions
2	Equal weight 0.2 across 5 dimensions	Composite became responsive to all dimensions; revealed that D3 was conflating two distinct signals (claim-level grounding vs citation-level correctness)	Kept equal weighting	Split D3 into two dimensions
3	Split D3 → D3 (factual_grounding, claim-level) + D6 (citation_correctness, citation-level)	A response with perfect IDs but wrong descriptions, and a response with weak IDs but correct substantive claims, both became scoreable distinctly	Adopted; rubric grew to 6 dimensions	Rebalance weights to sum
4	Six dimensions × equal weight 0.5 (composite normalized to 0–1)	Stable across the test corpus; composite range reflected actual response variation	Adopted as production rubric	Address judge-side reliability issues
5	JSON parse failures on long-context judge calls	Roughly 3–5% of calls produced malformed JSON; treating these as score-zero contaminated the model's measured score with judge infrastructure noise	Add JSON parse retries	Make retry rates observable
6	JSON retry implemented (default 3 attempts)	Most parse failures resolved within retry budget; residual failures concentrated on a small set of test cases with very long expected responses	Adopted	Surface retry rates for audit
7	Structured logging for retry events	Retry rate per run became visible in logs alongside test-case	Adopted	Decide how to handle

#	Iteration	Observation	Outcome	Motivated next
		progress; previously-silent failures became attributable to specific (judge_model, test_case) pairs		terminal retry-exhaustion cases
8	Skip-and-flag on terminal retry failure	Test case marked judge_error=True and excluded from aggregate metrics rather than scored zero; run summary reports skipped count separately	Adopted	Audit skipped cases per run for systemic patterns
9	Claim-level grounding refactor — D3 procedure changed from holistic 0–3 judgement to count-based ratio over the 5 most load-bearing claims	Inter-judge variance on D3 dropped substantially; per-judge "leniency" became less of a factor	Adopted	Apply count-based pattern to D6
10	Count-based ratio for D6 (CORRECT, IMPRECISE, MISATTRIBUTED, FABRICATED)	D6 became as reliable as D3 by the same mechanism — judge compares against deterministic verification output rather than holistic impression	Adopted	Compress D3 procedure to keep prompt tractable
11	Simplified D3 procedure (compressed from ~30 lines to ~15)	Long procedural blocks in the rubric had been confusing the judge — it occasionally followed a partial procedure and produced inconsistent counts. Compression to the essential steps recovered consistency	Adopted	Apply same compression to D4/D5
12	Anti-lenieny instruction in system prompt — "Leniency not based on ground truth is a scoring error"	Without this instruction, judge models drifted toward giving the benefit of the doubt to plausible-looking regulatory content; with it, scores on confidently-cited but unverifiable material dropped to expected ranges	Adopted	Final 6-dim equal-weight production rubric

12.4 Runtime Evaluation- Framework in Action

B3-001 B3_Conditional_Compliance_Reasoning PASS Bench: 1.00 RAGAs: 0.94 hybrid							
Question: A CIIO uses shared administrative accounts for managing CII systems, but has implemented comprehensive logging of all actions, real-time monitoring with alerts, and quarterly access reviews by an independent team. Given these compensating controls, does this setup comply with CCoP 2.0 multi-factor authentication requirements?							
Retrieved Citations: [1] CCoP 2.0::5.2.1 [2] CCoP 2.0::5.1.4 [3] CCoP 2.0::5.2.2							
Response (0 tokens, 0ms): **Initial Assessment** The CIIO uses shared administrative accounts for managing CII systems. Shared administrative accounts are generally discouraged as they can lead to security vulnerabilities if not properly managed. **Compliance Requirements** CCoP 2.0 mandates that the use of shared user accounts should be avoided unless absolutely necessary (Clause 5.2.1.c). It emphasizes the importance of minimizing privileges and monitoring all account activities. **Analysis of Implemented Controls** Despite using shared administrative accounts, the CIIO has implemented: - Comprehensive logging - Real-time monitoring with alerts - Quarterly access reviews by an independent team These are compensating controls that can mitigate risks associated with shared accounts if properly implemented. **Assessing Compliance with Multi-factor Authentication (MFA) Requirements** CCoP 2.0 requires multi-factor authentication for privileged accounts where privileges are escalated or accessed (Clause 5.3.1.c). Since the CIIO has not explicitly mentioned implementing MFA, this requirement is not fully met. **Alternative Solutions and Recommendations** Implementing MFA for all administrative logins would enhance compliance with CCoP 2.0 by adding an additional layer of security. The existing compensating controls should be retained and supplemented with MFA to address the identified gap. **Conclusion** While the CIIO has implemented effective monitoring and access review processes, the use of shared administrative accounts without multi-factor authentication does not fully comply with CCoP 2.0 requirements. The implementation of MFA is necessary to meet the specified standards. Sources: - CCoP 2.0, Clause 5.2.1 - CCoP 2.0, Clause 5.1.4 - CCoP 2.0, Clause 5.2.2							
Quality Breakdown by Benchmark							
Benchmark	ctx_recall	ctx_precision	ctx_faith	fct_recl	ans_relev	sem_sim	LLM Judge
B3: Conditional Compliance Reasoning	0.22	1.00	0.80	1.00	0.92	0.90	1.00

Figure 4: CCoP Evaluation framework in action, showing model response and evaluation for a RAG augmented pipeline, focusing on assessment of model grounding to CCoP facts